

HelixAgent

HelixAgent

به جای انتخاب یک مدل، بگذارید با هم بحث کنند و پاسخی را که بر سر آن توافق دارند ارائه دهند.

خلاصه

در AI و با قدرت (LLM) یک سرویس آماده‌ی تولید مبتنی بر مجموعه‌ای از مدل‌ها HelixAgent است که پاسخ‌های چندین مدل زبانی را به شکلی هوشمندانه ترکیب می‌کند — از جمله یک Go و انتخاب پویای ارائه‌دهندگان بر اساس تأیید — تا خروجی دقیق و AI سیستم بحث چندمرحله‌ای قابل اعتماد تولید کند.

توضیح کوتاه

است که چندین ارائه‌دهنده را در یک پاسخ Go یک سرویس مجموعه‌ای مبتنی بر HelixAgent را اجرا می‌کند، ارائه‌دهندگان را به صورت AI دقیق ادغام می‌کند. این سرویس بحث‌های چندمرحله‌ای امتیازدهی می‌کند، با استراتژی‌های مسیریابی مبتنی بر اطمینان LLMsVerifier پویا از طریق های به API هدایت می‌کند و ویژگی‌های تولیدی را ارائه می‌دهد: کش، نظارت، محافظ‌های امنیتی و OpenAI سبک.

توضیح مفصل

AI با قدرت (LLM) یک سرویس آماده‌ی تولید مبتنی بر مجموعه‌ای از مدل‌ها HelixAgent است که پاسخ یک مدل را به عنوان فرضیه در نظر می‌گیرد، نه حکم قطعی. به (MIT تحت مجوز)، جای تکیه بر یک ارائه‌دهنده که ممکن است اشتباه کند، سوگیری داشته باشد یا موقتاً در دسترس نباشد — پاسخ‌های چندین مدل زبانی را ترکیب می‌کند تا به دقیق‌ترین و قابل اعتمادترین خروجی دست یابد و وقتی سوالی به اندازه‌ی کافی دشوار باشد، مدل‌ها را در یک بحث ساختاریافته و چندمرحله‌ای قرار این سرویس، بسیاری از ارائه‌دهندگان README می‌دهد. فهرست ارائه‌دهندگان گسترده است: فایل Claude، مستند کرده است، از جمله internal/llm/providers/ را در مسیر LLM، DeepSeek، Gemini، Mistral، Qwen و xAI/Grok.

نکته‌ی کلیدی این است که انتخاب ارائه‌دهنده نه بر اساس فهرستی ثابت از اولویت‌ها، بلکه به صورت یکپارچه LLMsVerifier بلامرنگ و بر اساس عملکرد انجام می‌شود. امتیازهای تأییدزنده از یک مسیریابی و جایگزینی هوشمندانه با بهترین ارائه‌دهنده‌ی در حال کار را هدایت می‌کند و در صورت افت AI کیفیت یکی از ارائه‌دهندگان، گزارش خطای دسته‌بندی شده ارائه می‌دهد. هماهنگ‌کننده‌ی بحث اختلاف نظر‌ها را به سیگنال تبدیل می‌کند: از توپولوژی‌های مختلف (مش، ستاره، زنجیره) پشتیبانی

می‌کند و پروتکل مرحله‌ای منظمی دارد — پیشنهاد → نقد → بازبینی → ترکیب — با یادگیری بین‌بحثی تا سیستم در تطبیق مدل‌ها به مرور بهتر شود. استراتژی‌های مسیریابی شامل انتخاب مبتنی بر اطمینان، اجماع رأی اکثریت و تشخیص معنایی قصد کاربر است که همگی با جریان بلائرنگ پاسخ‌ها همراه هستند تا پاسخ‌ها به جای انتظار برای تکمیل کل مجموعه، توکن به توکن ارائه شوند.

این سرویس برای بقا در محیط تولید طراحی شده است، نه فقط نمایش خوب در دموها: Prometheus، یک لایه داده با دسترسی بالا را تشکیل می‌دهند و PostgreSQL و Grafana، معیارها، داشبوردها و ردیابی را فراهم می‌کنند و احراز هویت OpenTelemetry و JWT مجموعه را در چارچوبی از کنترل‌های لازم برای PII، محدودیت نرخ، موتور محافظها و تشخیص، استقرار واقعی قرار می‌دهند. این سرویس به صورت حدود بیست ماژول جداگانه سازماندهی شده است (EventBus، Observability، Auth، Storage، VectorDB، Embeddings، و غیره) که هر یک دغدغه‌ای مجزا را پوشش می‌دهند و یک چارچوب RAG، Memory، MCP را ارائه می‌دهد که با (کش معنایی، خروجی ساختاریافته، جریان پیشرفته) LLM بهینه‌سازی یکپارچه شده است. از SGLang، LlamaIndex، LangChain، Guidance و LMQL سازگار هستند، یک کلاینت موجود می‌تواند بدون OpenAI آنجا که نقاط پایانی تکمیل و مجموعه با متصل شود و از استدلال مجموعه‌ای بهره‌مند گردد HelixAgent نیاز به بازنویسی، به

محتوا

چرا آن را ساختیم

HelixAgent به‌تنهایی ممکن است نادرست، مغرضانه یا در دسترس نباشد LLM هر مدل ساخته شد تا برنامه‌ها بتوانند همزمان به چندین مدل مراجعه کنند، پاسخ‌های آن‌ها را بر اساس قابلیت اطمینان اندازه‌گیری شده بسنجند و با ظرفیت جایگزین کنند — و بدین ترتیب وابستگی شکننده به یک ارائه‌دهنده واحد را به مجموعه‌ای انعطاف‌پذیر و خودارزیاب تبدیل کنند.

چرا انقلابی است

این سامانه اجماع چندمدلی را عملیاتی می‌کند — یعنی «پرسیدن از چندین مدل و تطبیق پاسخ‌ها» را از اسکرپت‌های موقتی به یک سرویس تولیدی منتقل می‌کند. به‌جای کدنویسی سخت برای یک ارائه‌دهنده و امید به بهترین‌ها، تیم‌ها مسیریابی مبتنی بر امتیازهای تأییدزنده دریافت می‌کنند، پروتکل مناظره ساختاریافته برای پرسش‌هایی که یک بار پاسخگویی کافی نیست، و تاب‌آوری در سطح تولید OpenAI همه پشت یک — (لایه داده با دسترسی بالا، قابلیت رصد کامل و محافظ‌های امنیتی) دستاورد اصلی این است که بدون ایجاد اختلال در روند کار، وابستگی شکننده به یک API سازگار با ارائه‌دهنده به مجموعه‌ای انعطاف‌پذیر و خودارزیاب تبدیل می‌شود و مشتریان موجود تنها با تغییر یک به آن مهاجرت می‌کنند، نه تغییر کد (endpoint) نقطه پایانی.

نوآوری‌ها

- چندمرحله‌ای ساختاریافته که اختلاف‌نظر مدل‌ها را به‌عنوان منبعی ارزشمند تلقی AI مناظره می‌کند: توپولوژی‌های قابل انتخاب مش/ستاره/نجیره‌ای، پروتکل منظم پیشنهاد→نقد→بازبینی→ترکیب، و یادگیری بین‌مناظره‌ای که با گذشت زمان انباشته می‌شود.
- به‌جای فهرست ترجیحی LLMsVerifier انتخاب پویای ارائه‌دهنده بر اساس امتیازهای زنده ثابت — مجموعه به سمتی هدایت می‌شود که در حال حاضر عملکرد بهتری دارد و در صورت افت کیفیت یک ارائه‌دهنده، با ظرفیت جایگزین می‌شود.
- کش معنایی، خروجی ساختاریافته، جریان‌سازی LLM برای Go چارچوب بهینه‌سازی بومی (SGLang) پیشرفته که به‌تنهایی کل‌آمد است و بهینه‌سازهای خارجی اختیاری

را می‌توان در صورت نیاز به آن (LlamaIndex، LangChain، Guidance، LMQL) افزود، نه اینکه اجباری باشند.

- معماری مدولار متشکل از حدود بیست ماژول جداگانه که جداسازی دغدغه‌ها را ممکن می‌سازد و راه را برای ویژگی‌های کلان‌داده مانند حافظه توزیع شده و جریان‌سازی گراف دانش باز می‌کند

بزرگ‌ترین چالش‌های فنی و راه‌حل‌های ما

- **رائه‌دهندگان** از نظر کیفیت متفاوت‌اند و با گذشت زمان، انتخاب از میان **رائه‌دهندگان** نابرابر دچار تغییر می‌شوند، بنابراین هر رتبه‌بندی ثابتی فردا نادرست خواهد بود. ما این مشکل را با به مسیریابی **LLMsVerifier** انتخاب مبتنی بر اندازه‌گیری مستمر حل کردیم: امتیازهای مبتنی بر رأی اکثریت و وزن‌دهی اعتماد تغذیه می‌شوند و در صورت افت کیفیت یک **رائه‌دهنده**، به جای اعتماد به آن، با ظرفیت جایگزین می‌شود.
- یک مدل تنها، در یک بار پرسش، دریافت پاسخ قابل اعتماد برای پرسش‌های واقعاً دشوار — مکانیزمی برای تشخیص خطای خود ندارد. هماهنگ‌کننده مناظره این مکانیزم را فراهم می‌کند. مناظره چندتوپولوژی و مرحله‌ای (پیشنهاد → نقد → بازبینی → ترکیب) که مدل‌ها را وادار می‌کند، پیش از ارائه پاسخ نهایی، یکدیگر را به چالش بکشند و اصلاح کنند.
- ارسال پرسش به چندین **اجرای مجموعه‌ای از مدل‌ها در محیط تولید**، نه فقط در یک نوت‌بوک **رائه‌دهنده** سطح شکست را چند برابر می‌کند. ما این مشکل را با لایه داده **PostgreSQL+Redis** با دسترسی بالا، قابلیت رصد **Prometheus/Grafana/OpenTelemetry** برای تشخیص رفتار نادرست **رائه‌دهنده** یا مسیر، و محیط امنیتی شامل **JWT** احراز هویت **PII** محدودیت نرخ، موتور محافظ‌ها و تشخیص، **مهار** کردیم.

محتوا

پشته فناوری

- انتخاب شده زیرا ارسال یک درخواست به چندین **رائه‌دهنده** به‌طور همزمان دقیقاً همان — **Go** کاری است که گوروتین‌ها برای آن طراحی شده‌اند، و استقرار تک‌باینری باعث می‌شود سرویس با حدود ۲۰ ماژول به‌سادگی قابل ارسال باشد؛ این فناوری زیربنای کل سرویس و هر ماژول داخلی است.
- **Gin** (وب **API**) — سریع و کم‌بار؛ این فناوری **HTTP** انتخاب شده برای ارائه یک سطح — **OpenAI** نقاط پایانی سازگار با چت، استریم و تجمیع را ارائه می‌دهد که به **v1** برای تکمیل **OpenAI** نقاط پایانی سازگار با کاربران موجود امکان می‌دهد بدون تغییر از تجمیع استفاده کنند.
- **PostgreSQL** — انتخاب شده به‌عنوان ذخیره‌ساز پایدار برای جلسات، تحلیل‌ها و سوابق — مناظره‌ها، تا تصمیمات اجماع و تاریخچه مناظره‌ها قابل حسابرسی باشد؛ این فناوری لایه داده با دسترسی بالا را تثبیت می‌کند.
- **Redis** — انتخاب شده برای کشینگ کم‌تأخیر و صف‌بندی وظایف؛ این فناوری هم کشینگ — پاسخ‌ها و هم لایه کش معنایی را تقویت می‌کند که به درخواست‌های تکراری یا تقریباً تکراری اجازه می‌دهد از استنتاج‌های زائد عبور کنند.
- **LLMsVerifier** (یکپارچه) — انتخاب شده تا قابلیت اطمینان **رائه‌دهندگان** به جای یک فرض، کمیتی قابل اندازه‌گیری باشد؛ امتیازهای آن **رائه‌دهندگان** را برای مسیریابی رتبه‌بندی کرده و در صورت افت کیفیت یکی از آن‌ها، جایگزینی را فعال می‌کند.
- **Prometheus + Grafana + OpenTelemetry** — انتخاب شده تا تجمیعی که چندین **رائه‌دهنده** را در بر می‌گیرد، قابل مشاهده باقی بماند؛ این فناوری معیارهای داشبوردها و ردیابی درخواست‌های سرتاسری در فرآیند پخش را نمایش می‌دهد، **helixagent_***

- انتخاب شده برای قابلیت — **Model Context Protocol (MCP)** آداپتورهای برای اتصال MCP فهرستی از آداپتورهای README توسعه از طریق یک پروتکل باز؛ فایل ابزارها و زمینه‌های خارجی را ارائه می‌دهد.
- **Neo4j / ClickHouse / Kafka** (کلان‌داده) — انتخاب شده برای عبور از حافظه توزیع شده و ویژگی‌های گراف دانش را ClickHouse و Neo4j: محدودیت تک‌نود. این گراف و داده‌های رویدادها در مقیاس بزرگ استریم می‌کند Kafka پشتیبانی می‌کنند، و
- **SGLang, LlamaIndex, LangChain, Guidance, LMQl** — انتخاب شده برای افزودن کش پیشوندی، بازیابی، تجزیه وظایف و تولید محدود — به‌عنوان سرویس‌های اختیاری، تا بهینه‌سازی‌های سنگین‌تر بدون اجبار در دسترس باشند.

وضعیت و یادداشت‌های شفافیت

- این سرویس به‌عنوان آماده تولید توصیف شده است، اما ارقام عملکرد و پوشش در **وضعیت: بتا** مانند «بیش از ۱۰۰۰ درخواست در ثانیه»، «کمتر از ۵۰۰ میلی‌ثانیه برای README فایل درخواست‌های کش‌شده»، تعداد ارائه‌دهندگان و اسکرپیت‌های اعتبارسنجی) ادعاهای خود پروژه هستند و به‌طور مستقل تأیید نشده‌اند و در اینجا عمداً کیفی نگه داشته شده‌اند.
- متفاوت است؛ این صفحه از چارچوب کیفی README تعداد ارائه‌دهندگان در خود فایل. تعداد زیادی ارائه‌دهنده» استفاده می‌کند.

اصلی-Helix: اولویت لایه